# Group Project 2: Primer Finder

- due Friday June 5th by class time
- 20 pts (see rubric on final page)
- to be submitted with peer reviews (4 of the 20 pts)

## Objectives

- To implement a program that can determine suitable *primers* for **degenerate PCR**
- To practice with *degeneracy*
- To participate and communicate well in a team

## Team Dynamics

You will get the most out of this project if you put effort into working as a team: communicating with your colleagues early and often. There are parts of this project that are really only understandable in-depth by the biologists, and parts that are much more easily grasped by those who have previously taken a computer programming course. That's deliberate. Each of you now has a basic understanding of the other's field, and you should be helping each other understand new concepts. The more you do that, the more effective everyone on the team will be in contributing to a complete project.

## Your Mission

In this final project, your team will build a program, `primer_finder.py`, with a function, `primer_finder(<file>)`, that will read an input file containing a multiple sequence alignment (MSA). From that MSA, your program will determine appropriate pairs of **upstream and downstream primers** that can be used in the Polymerase Chain Reaction (PCR) method of gene amplification. The output of your program should be:

1. A reprint of the input MSA in a "pretty" consensus-sequence format. For example, if the MSA consisted of the two sequences "KYTQ" and "RYSQ", the reprint would be:

```
K/R Y T/S Q
```

2. a summary list of "good" candidate upstream and downstream primers that your algorithm found. A primer is considered "good" if it meets *all* of the following criteria:

    - the primer should be based on 7-9 consecutive columns in the MSA. This range should be adjustable in your program.

- the primer should be based on a place where there is a lot of agreement in the MSA. This corresponds to a lot of consecutive "black columns" in the output as we've seen in class. We will simulate this by saying that if a primer is "good", at least 70% of its columns will have complete identity (a single amino acid that all sequences agree upon). For example, in order for an 8-column sequence to remain a candidate, at least 6 out of the 8 columns must be identities, since 5 out of 8 is < 70%. This percentage should be adjustable in your program.
- it has a non-degenerate 3′ end. This means (1) the last two amino acids upon which the primer is based do not contain any of the high-degeneracy amino acids that have six different nucleotide encodings (R, S, or L). **AND** (2), the final column (nucleotide) of the primer must be an identity (no degeneracy). **PITFALL ALERT:** the 3′ end differs depending on whether it is an upstream or a downstream primer!
- In addition to not having R, S, or L in the last two columns, there can't be more than two of these "bad" amino acids *anywhere* in the primer.

In your summary list, primers should be printed by their position numbers. For example, `[10, 16]` indicates a 7-column primer starting at amino acid #10 and ending at amino acid #16 of the MSA (including the end positions). **Remember, the first residue will be amino acid #0**. Here's a sample primer summary:

```
upstream candidates:
[[6, 12], [7, 13], [8, 14], [20, 26], [21, 27], [45, 51]]
downstream candidates:
[[6, 12], [10, 16], [23, 29], [45, 51]]
```

List downstream candidates the same way as upstream candidates: in left-to-right order by position.

3. A detail blurb about each of the upstream primers printed above. For each upstream primer, you should show:

   - a reprint of the position numbers of that primer
   - the amino acids that make up the primer (in a "pretty" format)
   - the nucleotide sequence of the primer (in a "pretty" format). Please substitute "N" for A/T/C/G but leave all other combinations as-is.
   - the number of nts deleted from the 3′ end to reduce degeneracy (this will be either 0 or 1)
   - the overall degeneracy of the primer

For example, here are the details for two possible upstream primers:

```
UPSTREAM
--------
6...12
amino acids: K A L L/I I G I
nucleotides: A A A/G G C N T/C T N T/C/A T N A T A/C/T G G N A T
deleted 1 nt from 3' end
degen: 9216

7...13
```

```
amino acids: A L L/I I G I N
nucleotides: G C N T/C T N T/C/A T N A T A/C/T G G N A T A/C/T A A
deleted 1 nt from 3' end
degen: 13824
```

4. A detail blurb about each of the downstream primers printed above. For each downstream primer, you should show the same info as for the upstream primers except for two changes. First, you should not need to delete nucleotides from the "initial" 3′ end of downstream primers. (Make sure you know why!) Second, don't forget that the nucleotide sequence for downstream primers is the reverse complement of what the upstream version would be. Please print the amino acids in "normal" left-to-right order (N-terminus to C-terminus), but then print the **nucleotide** sequence in reverse complement order. For example, here is the detail for a possible downstream primer:

```
DOWNSTREAM
----------
10...16
amino acids: I G I N Y L/F/I G/N
nucleotides: N T/C T/C N A T/G/A A/G T A A/G T T A/G/T A T N C C A/G/T A T
degen: 27648
```

5. Finally, your program should print a list of good primer pairs. A pair of primers is "good" if **all three** of the following criteria are met: (1) the pair consists of one upstream and one downstream primer, (2) the downstream primer is "to the right of" the upstream primer when viewing the MSA, and (3) the number of amino acids separating the 5′ end of the upstream primer from the 5′ end of the downstream primer is more than 35. This threshold (35) should be easily changeable in your program. Here is a sample printout of this section:

```
GOOD PRIMER PAIRS
-----------------
upstream:  [6, 12] K A L L/I I G I
downstream:  [45, 51] D M/I V I L/M T D

upstream:  [7, 13] A L L/I I G I N
downstream:  [45, 51] D M/I V I L/M T D
```

## Adjustables

As alluded to above, your program should contain the following easily-adjustable constants to make it more versatile:

- `MIN_LENGTH` and `MAX_LENGTH`. These two control the length of the primer (in amino acids). Set these to 7 and 9, respectively, which corresponds to 21-27 nts.
- `AGREEMENT_THRESHOLD`. This controls the % of columns that must be identities in the primer. Set this to 70% for your tests.
- `FILE` for the name of the input file.
- `PAIR_DISTANCE`. This tells how far apart the upstream and downstream primers must be, at minimum. Set to 35 for your tests.

## Starter Files

To help you out, the following starter files oare on the course website

- `CodonList.py`. This Python file contains a function that, given an amino acid, returns a list of codons that encode for that amino acid. Feel free to import or copy this function into your own project.
- Two sample input files, (of course, your program should work with any valid input file. (See the **Testing** section below.) Each of these contains an MSA upon which primers will be based. The form of an input file is:

```
<number of sequences in MSA>
<sequence 1>
<sequence 2>
<sequence 3>
etc.
```

## Algorithm Details

Here is a basic algorithm for this program. You are not required to follow this algorithm, but it may help you to organize your thoughts.

1. Read in the input file into a form you can process. A "list-of-lists" format works well. If you are unfamiliar with how to read data from an input file in Python, be sure to read the Python online book.
2. Find all possible candidate "windows" (like columns `[10,16]` of the MSA) that meet the length criterion, the `AGREEMENT_THRESHOLD` criterion, and the `low-degenerate-amino-acid` criterion.
3. From the list of windows you made in the last step, find all good candidate upstream and downstream primers that meet the non-degenerate-3'-end criterion. Remember, `[10,16]` makes a good upstream candidate if position #16 is non-degenerate. But the same window makes a good downstream candidate only if position #10 is non-degenerate!
4. Pretty print the input MSA, the summary list, and the detail blurbs.
5. Compute good pairs from the upstream and downstream lists you made previously. Print them.

All group members should be able to understand the algorithm that your program uses, even if it is the computer science member(s) of your team that are doing the bulk of the coding. Each biologist on the team must write at least one function of this project. Please credit who wrote each function in the comment that precedes the function's definition line. Good candidate functions that biologists should be able to write include:

- a function that prints nucleotide (or protein) sequences in a "pretty" way
- a function that turns an nt sequence into its reverse complement
- a function that computes degeneracy

Exactly what functions you need to write is up to you, so there could be other suitable ones depending on how you organize your program.

## Testing

You should first test your program with the two sample input files. One is small enough that you can figure out the results by hand. (Do that! Just because your program outputs numbers surrounded by brackets doesn't mean they are right.) Your goal is to design degenerate PCR primers that will enable you to isolate a PCR product from a related organism, which we don't need to specify here.

In your next test, your team will also have to generate a unique MSA input file consisting of a minimum of 8 protein sequences aligned to each other. Which sequences you pick is entirely up to you - pick genes that encode proteins whose function is of biological interest to your team.

You should also designate a ninth "test" organism (Genus and species) that does not presently have the gene of interest isolated, and is therefore different from the species whose sequences are in your MSA. **This test organism will not be part of your alignment (because you haven't isolated the gene or protein yet!) but it will act as an experimental goal that will (hopefully) justify why you picked the sequences you did. Please ask if this doesn't make sense!** This organism should be closely related to at least some (if not most or even all) of the species whose protein sequences are used in your alignment. Please specify in your write-up your rationale for picking the eight sequences you did for your MSA, and why you chose the ninth test organism you did.

Remember, the goal here is to design primers that would ultimately be used in a degenerate PCR experiment to isolate a homologous gene fragment. It is important that the alignment you come up with is an appropriate and suitable input for your program to be able to find "good" degenerate oligonucleotide primers. A certain minimum amount of sequence conservation (7 amino acids) in at least two regions of your aligned sequences will be necessary. These conserved regions should also be mostly free of the degenerate amino acids described earlier. Remember, you don't need to have a consecutive set of completely conserved amino acid residues (i.e. "all black" in multiple consecutive columns) in order to design a "good" degenerate primer. Sometimes a grey (similar) or even white (non-conserved) amino acid imbedded in the middle of a string of "blacks" will not present a major problem in primer design. You can also "cheat" (only if necessary) in the design of your primers. What is meant by cheating is that in cases of less-than-unanimous identity (e.g. 5 blacks and 1 white or grey in a single column of your MSA), you can ignore the outlier - especially if you have reason to believe that the organism from which the template genomic DNA is derived in your hypothetical PCR experiment is less closely related to this outlier than to the other species in your alignment.

Before you use your MSA as input for your program, you must do two things:

1. Go through the alignment "by eye" (like we did in class), and try and design *at least one* upstream/downstream degenerate primer *pair*. You should use Clustal Omega and ESPript for this process. The results will be helpful to make sure that the MSA is suitable input for your program.
2. Once you have completed the manual run-through, you should take your ESPript results and your "by eye" primer set and run these by Prof. Horton, so he can vet them and make sure that everything is OK. You should be working on your program even before this vetting process is done; *i.e.* use the sample input file to test your program, *even before* trying your own MSA input. Once Prof. Horton has given you the go-ahead with your MSA, you should then use it as input for your program and see what primer sets it selects for use in your hypothetical PCR experiment.

## Is this biologically sound?

Finally, as in the previous project, you should analyze the results of your tests and produce a write-up that reflects upon the strengths and weaknesses of the program you have written. Answer the following questions:

1. Was the primer pair you found "by eye" in your sample input also found by your program? Explain either way (if it did – why? If it did not – why not?).
2. Are there any primers that your program found that would *not* be suitable for the PCR process? If so, *why* aren't they suitable? If all are suitable, what makes them so?
3. Are there any suitable primers that the biologists in your group can find in your sample input "by eye" that your program did *not* find? Why are those that you found "by eye" still suitable? Again, use ESPript to help you find primers manually.
4. If you did find suitable ones "by eye" in #3, above, what is it about your program that kept them from being found? Speculate on possible improvements to your program. Answers to these questions are not the only things we expect to see in your final report. We also expect to see your MSAs, your ESPript results, your primers with degeneracy data, and discussion about your answers to the above questions.

### How to turn this in

Please turn in all Python files, including `primer_finder.py`, and the write-up named `primer_finder.pdf` on gradescope. Individually submit peer reviews. Don't forget to put the honor code affirmation in the comments of both.

| | Exceeding expectations (1x) | Arriving at expectations (1.5x) | Not meeting Expectations (0.5x) |
|---|---|---|---|
| CS elements (6 pts) | Correct primer selection | Partially correct primer selection | Incorrect primer selection |
| | Improvements made to the algorithm and explained | Improvements made to the algorithm, not explained | No improvements made to the algorithm |
| | Detailed and complete blurb and pretty printing | Detailed and complete blurb xor pretty printing | Neither complete blurb nor pretty printing |
| Bio elements (6 pts) | Appropriate test data | Some appropriate test data | Insufficient appropriate test data |
| | Conclusions regarding biological soundness | Conclusions partially regarding biological soundness | Conclusions not regarding biological soundness |
| Reflective (8 pts) | Thorough and understandable write-up of results | Thorough xor understandable write-up of results | Neither thorough anor understandable write-up of results |
| | Thorough and reflective write-up of team dynamics (peer review) | Thorough xor reflective write-up of team dynamics (peer review) | Neither thorough nor reflective peer reviews |